



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

INGENIERÍA EN COMPUTACIÓN

Laboratorio de Organización de Arquitectura de Computadoras

Reporte - Diseño de máquinas de estado

NOMBRE DE LOS INTEGRANTES:

Omar Alfonso Zendejas Labias Grupo de Teoría: 04

Emiliano García Olvera Grupo de Teoría: 02

NOMBRE DEL PROFESOR: ING. KAREN PÉREZ

GRUPO: 06 EQUIPO: 9

SEMESTRE: 2027-1.

FECHA DE ENTREGA: 15 de septiembre de 2026

Objetivo

Se busca analizar, diseñar e implementar una máquina de estados finitos a partir de una carta ASM proporcionada, identificando el funcionamiento de los *flip-flops* tipo D como elementos de memoria. A partir del llenado de la tabla de transiciones y de la reducción mediante mapas de Karnaugh se obtienen las funciones de estado siguiente y de salida, las cuales posteriormente se implementan en VHDL y se verifican sobre la tarjeta de desarrollo FPGA DE10-Lite.

Introducción

Una carta ASM (*Algorithmic State Machine*) es una representación gráfica del comportamiento de un circuito secuencial, equivalente a un diagrama de estados, en la que cada estado se dibuja como una caja rectangular y cada condición de transición como una caja de decisión. A diferencia del diagrama de estados clásico, la carta ASM se asemeja a un diagrama de flujo y resulta más directa para derivar, a partir de ella, la tabla de transiciones del sistema.

En una implementación síncrona, el estado de la máquina se almacena en un conjunto de *flip-flops* tipo D, que actúan como elementos de memoria: en cada flanco de *clock* capturan el valor calculado como estado siguiente y lo mantienen hasta el siguiente flanco. Las entradas *D* de estos *flip-flops*, así como las señales de salida de la máquina, se obtienen como funciones booleanas del estado actual y de las entradas del sistema; dichas funciones se simplifican mediante mapas de Karnaugh a partir de la tabla de transiciones, siguiendo el procedimiento estándar de diseño de circuitos secuenciales síncronos descrito, entre otros, por Savage Carmona *et al.* [1].

Desarrollo

Parte I. Análisis y diseño de la máquina de estados

La carta ASM proporcionada para esta práctica se muestra en la figura 1.

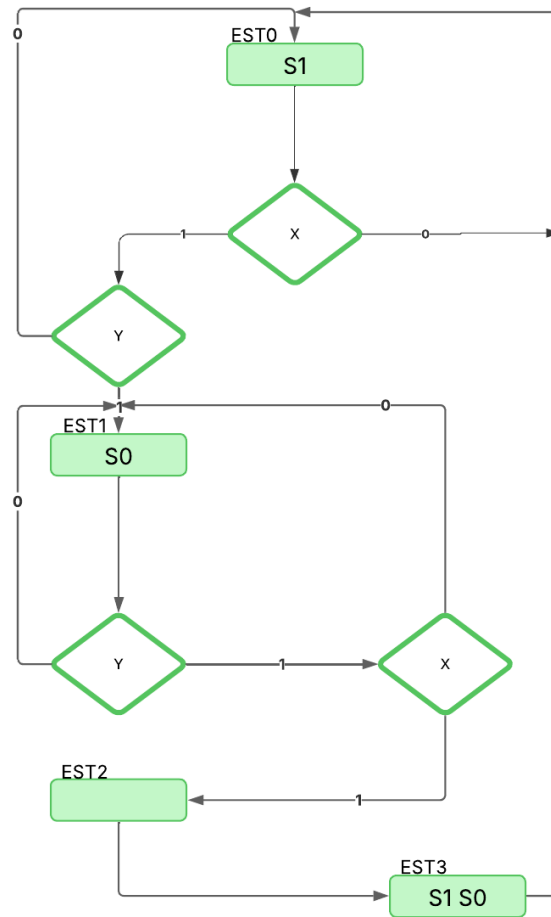


Figura 1: Carta ASM del sistema, proporcionada en el manual de la práctica.

3.1.1. Identificación de estados, entradas y salidas

La carta ASM define cuatro estados: EST0, EST1, EST2 y EST3. La máquina tiene dos entradas, X y Y , y dos señales de salida, S_1 y S_0 , cuyo valor depende únicamente del estado actual (es decir, se trata de una máquina de tipo Moore): EST0 activa S_1 ($S_1S_0 = 10$), EST1 activa S_0 (01), EST2 no activa ninguna salida (00) y EST3 activa ambas (11).

3.1.2. Codificación de los estados

Al ser cuatro estados se requieren 2 bits, almacenados en dos flip-flops tipo D con salidas Q_1Q_0 . Se propuso la codificación secuencial: EST0= 00, EST1= 01, EST2= 10, EST3= 11.

3.1.3. Tabla de transiciones

Siguiendo las rutas de decisión de la carta ASM (figura 1) se obtiene la siguiente tabla de transiciones, en la que un asterisco indica que la salida de ese estado no depende de dicha entrada:

Estado actual		Entradas		Estado siguiente		Salidas	
Edo.	Q_1Q_0	X	Y	Edo.	Q_1Q_0	S_1	S_0
EST0	00	0	*	EST0	00	1	0
EST0	00	1	0	EST0	00	1	0
EST0	00	1	1	EST1	01	1	0
EST1	01	*	0	EST1	01	0	1
EST1	01	0	1	EST1	01	0	1
EST1	01	1	1	EST2	10	0	1
EST2	10	*	*	EST3	11	0	0
EST3	11	*	*	EST0	00	1	1

Tabla 1: Tabla de transiciones obtenida a partir de la carta ASM.

3.1.4. Mapas de Karnaugh

A partir de la tabla 1 se construyeron los mapas de Karnaugh de las entradas D_1 , D_0 de los flip-flops y de las salidas S_1 , S_0 , usando como ejes Q_1Q_0 y XY :

$Q_1Q_0 \backslash XY$	00	01	11	10	$Q_1Q_0 \backslash XY$	00	01	11	10
00	0	0	0	0	00	0	0	1	0
01	0	0	1	0	01	1	1	0	1
11	0	0	0	0	11	0	0	0	0
10	1	1	1	1	10	1	1	1	1
D_1					D_0				
$Q_1Q_0 \backslash XY$	00	01	11	10	$Q_1Q_0 \backslash XY$	00	01	11	10
00	1	1	1	1	00	0	0	0	0
01	0	0	0	0	01	1	1	1	1
11	1	1	1	1	11	1	1	1	1
10	0	0	0	0	10	0	0	0	0
S_1					S_0				

Figura 2: Mapas de Karnaugh de D_1 , D_0 , S_1 y S_0 . En D_1 , el grupo azul (renglón $Q_1Q_0 = 10$ completo) da el término $Q_1\overline{Q_0}$ y la celda naranja aislada da $\overline{Q_1}Q_0XY$. En D_0 se agrupan el renglón $Q_1Q_0 = 10$ ($\overline{Q_1}\overline{Q_0}$), la celda $Q_1Q_0 = 00, XY = 11$ ($\overline{Q_0}XY$), y dentro del renglón $Q_1Q_0 = 01$ las columnas $XY = 00, 01$ ($\overline{Q_1}Q_0\overline{X}$) y $XY = 00, 10$ ($\overline{Q_1}Q_0\overline{Y}$). En S_1 los dos renglones llenos dan $\overline{Q_1}\overline{Q_0}$ y Q_1Q_0 . En S_0 los renglones $Q_1Q_0 = 01$ y 11 forman un solo grupo de ocho celdas: Q_0 .

3.1.5. Funciones booleanas simplificadas

De los mapas de Karnaugh de la figura 2 se obtienen las siguientes funciones simplificadas:

$$D_1 = Q_1 \overline{Q_0} + \overline{Q_1} Q_0 XY \quad (1)$$

$$D_0 = Q_1 \overline{Q_0} + \overline{Q_0} XY + \overline{Q_1} Q_0 \overline{X} + \overline{Q_1} Q_0 \overline{Y} \quad (2)$$

$$S_1 = \overline{Q_1} \overline{Q_0} + Q_1 Q_0 \quad (3)$$

$$S_0 = Q_0 \quad (4)$$

3.1.6. Circuito resultante

El circuito de la figura 4 (sección Parte II), obtenido al compilar el proyecto en Quartus Prime, implementa directamente las ecuaciones 1-4 mediante compuertas AND, OR y NOT que alimentan las entradas D de dos flip-flops tipo D.

Parte II. Implementación en Quartus y FPGA DE10-Lite

Como base para generar la señal de reloj lenta necesaria para observar las transiciones de la máquina de estados en la FPGA, la profesora proporcionó el módulo divisor de frecuencia `div_frec.vhd`, mostrado en la figura 3.

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6 entity div_frec is
7     Port (reloj : in std_logic;
8           div_clk : out std_logic);
9 end div_frec;
10
11 architecture Behavioral of div_frec is
12 begin
13     process (reloj)
14         variable cuenta : std_logic_vector (27 downto 0) := X"00000000";
15         begin
16             if rising_edge (reloj) then
17
18                 --if cuenta 0 "40000000" then
19                 if cuenta = X"2FAF080" then
20                     cuenta := X"00000000";
21                 else
22                     cuenta := cuenta+1;
23                 end if;
24             end if;
25             div_clk <= cuenta(25);
26         end process;
27     end Behavioral;

```

Figura 3: Módulo `div_frec.vhd`, proporcionado por la profesora como base del proyecto.

Este módulo fue el punto de partida entregado por la profesora para generar la señal de reloj lenta. La lógica de la máquina de estados propiamente dicha se capturó como diagrama esquemático (`practica2_ff.bdf`) dentro del mismo proyecto de Quartus, integrando las compuertas que implementan las ecuaciones 1-4 junto con `div_frec.vhd`, como se detalla en la sección de Implementación.

3.2.1. Creación del proyecto en Quartus

Se creó el proyecto `practica2_ff` en Quartus Prime Lite Edition 18.1 para el dispositivo MAX 10 - 10M50DAF484C7G de la tarjeta DE10-Lite, tal como lo especifica el manual (figura 6).

3.2.2. Diagrama lógico e integración con `div_frec.vhd`

La figura 4 muestra el esquemático compilado del proyecto: el módulo `div_frec` (parte inferior) recibe el reloj de la tarjeta y entrega la señal de reloj lenta que alimenta a los dos flip-flops tipo D de

la máquina de estados; las entradas PIN_C10/PIN_C11 (X, Y) y la red de compuertas AND/OR/NOT que implementa las ecuaciones 1-4 producen las salidas S1 y S0.

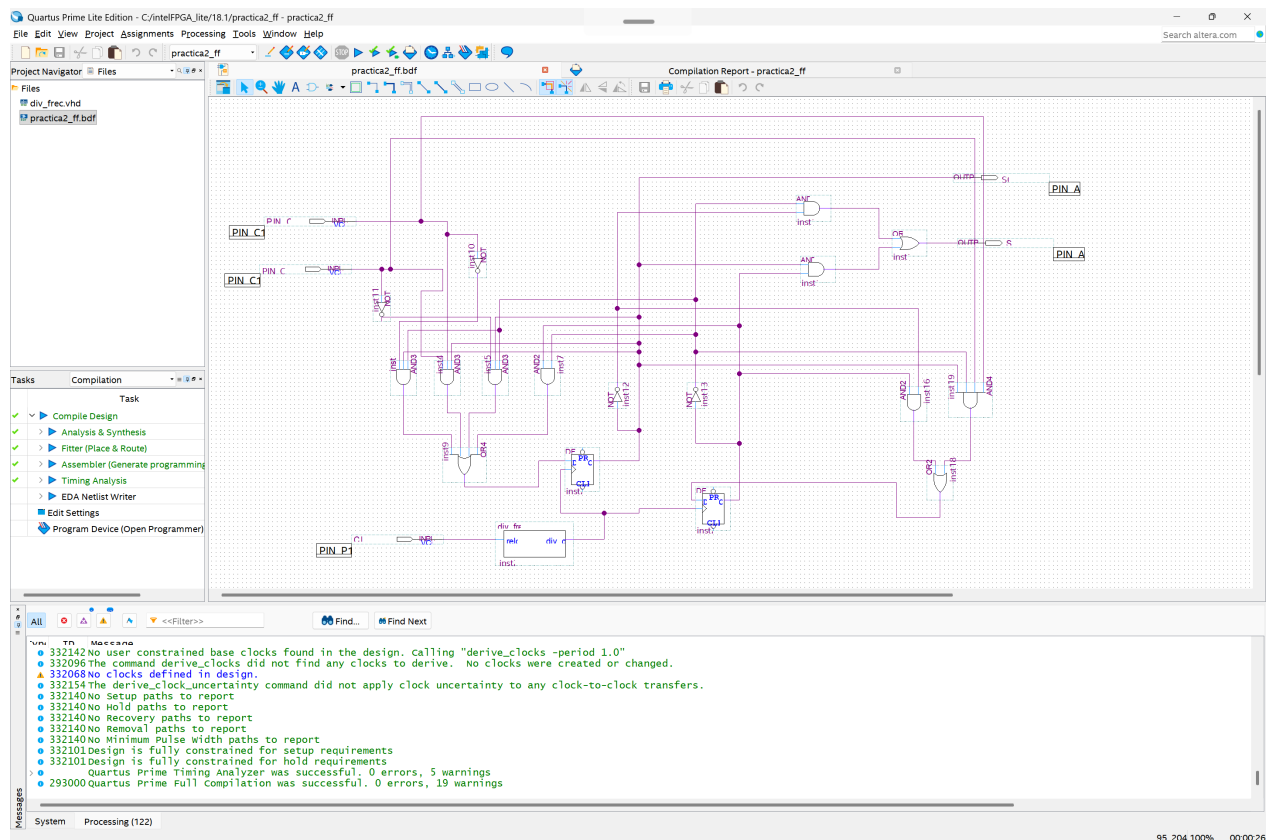


Figura 4: Esquemático compilado (practica2_ff.bdf) en Quartus Prime, integrando div_freq con la lógica de la máquina de estados.

3.2.3. Implementación

Capturamos la lógica del estado siguiente y de salida como diagrama esquemático (.bdf) con compuertas individuales dentro de Quartus Prime, en lugar de escribir un módulo VHDL equivalente para la máquina de estados. Ambas representaciones son funcionalmente equivalentes, ya que el esquemático implementa directamente las ecuaciones 1-4; el único módulo en VHDL del proyecto es el divisor de frecuencia proporcionado por la profesora.

Como el manual pide explícitamente la implementación en VHDL, a continuación se incluye el código equivalente de la máquina de estados, escrito directamente a partir de las ecuaciones 1-4 obtenidas en la Parte I. Este código se presenta únicamente como documentación de dicha implementación: no se integró al proyecto de Quartus ni se compiló, por lo que no sustituye al esquemático de la figura 4, que es el que efectivamente se programó en la tarjeta.

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3
4 entity maquina_estados is
5     Port ( clk : in  STD_LOGIC;
6           X   : in  STD_LOGIC;
7           Y   : in  STD_LOGIC;
8           S1  : out STD_LOGIC;
9           S0  : out STD_LOGIC);
10 end maquina_estados;
11
12 architecture Behavioral of maquina_estados is
13     signal Q1, Q0 : STD_LOGIC := '0';
14     signal D1, D0 : STD_LOGIC;
15 begin
16
17     -- Estado siguiente (ecuaciones 1 y 2)
18     D1 <= (Q1 and not Q0) or (not Q1 and Q0 and X and Y);
19     D0 <= (Q1 and not Q0) or (not Q0 and X and Y)
20           or (not Q1 and Q0 and not X) or (not Q1 and Q0 and not Y);
21
22     -- Flip-flops tipo D: capturan el estado siguiente en cada flanco de
23     subida
24     process (clk)
25     begin
26         if rising_edge(clk) then
27             Q1 <= D1;
28             Q0 <= D0;
29         end if;
30     end process;
31
32     -- Salidas (ecuaciones 3 y 4)
33     S1 <= (not Q1 and not Q0) or (Q1 and Q0);
34     S0 <= Q0;
35 end Behavioral;

```

Figura 5: Código VHDL de la máquina de estados, escrito a partir de las ecuaciones 1-4 de la Parte I. Entrada de clk: en la integración con div_frec.vhd, correspondería a la señal div_clk (reloj ya dividido), no al reloj de 50 MHz de la tarjeta.

3.2.4. Asignación de pines

La asignación de entradas y salidas a los recursos físicos de la DE10-Lite, tomada del *Pin Planner* de Quartus, se muestra en la tabla 2 y en la figura 6.

Señal	Dirección	Pin	Descripción
CLK	Entrada	PIN_P11	Reloj de la tarjeta (50 MHz)
PIN_C10	Entrada	PIN_C10	Entrada X (SW0)
PIN_C11	Entrada	PIN_C11	Entrada Y (SW1)
S0	Salida	PIN_A8	Salida S_0 (LEDR0)
S1	Salida	PIN_A9	Salida S_1 (LEDR1)

Tabla 2: Asignación de pines del proyecto practica2_ff.

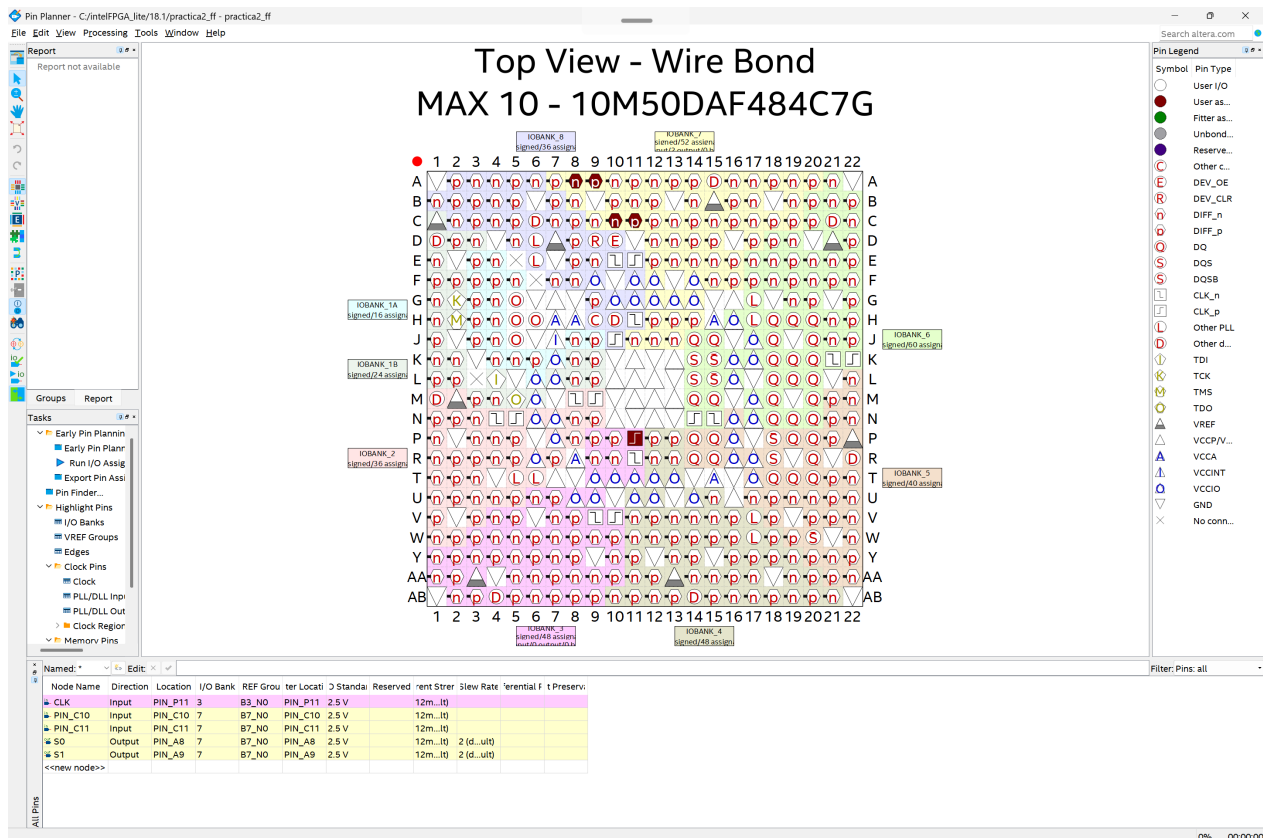


Figura 6: Pin Planner de Quartus Prime mostrando el dispositivo (MAX 10 – 10M50DAF484C7G) y la tabla de asignación de pines.

De acuerdo con la hoja de pines oficial de la DE10-Lite, PIN_C10 y PIN_C11 corresponden a los interruptores deslizables (*slide switches*) SW0 y SW1 de la tarjeta, utilizados como entradas X y Y respectivamente; mientras que PIN_A8 y PIN_A9 corresponden a los LEDs rojos LEDR0 y LEDR1, utilizados como salidas S_0 y S_1 respectivamente.

3.2.5. Compilación y programación de la FPGA

El proyecto compiló sin errores: el reporte de compilación de la figura 4 indica “*Quartus Prime Full Compilation was successful. 0 errors, 19 warnings*”. La FPGA se programó y se comprobó su funcionamiento físicamente, como se muestra en la figura 7.

Las 19 advertencias reportadas por Quartus corresponden típicamente a pines o señales sin restricciones de temporización (*timing constraints*) explícitas, como se aprecia en el registro de mensajes de la figura 4, y no afectan el funcionamiento del diseño al tratarse de un circuito secuencial simple sin requisitos críticos de *timing*.

3.2.6. Comprobación experimental

Se tomaron fotografías de la tarjeta DE10-Lite programada con distintas combinaciones de las entradas X, Y (figura 7).

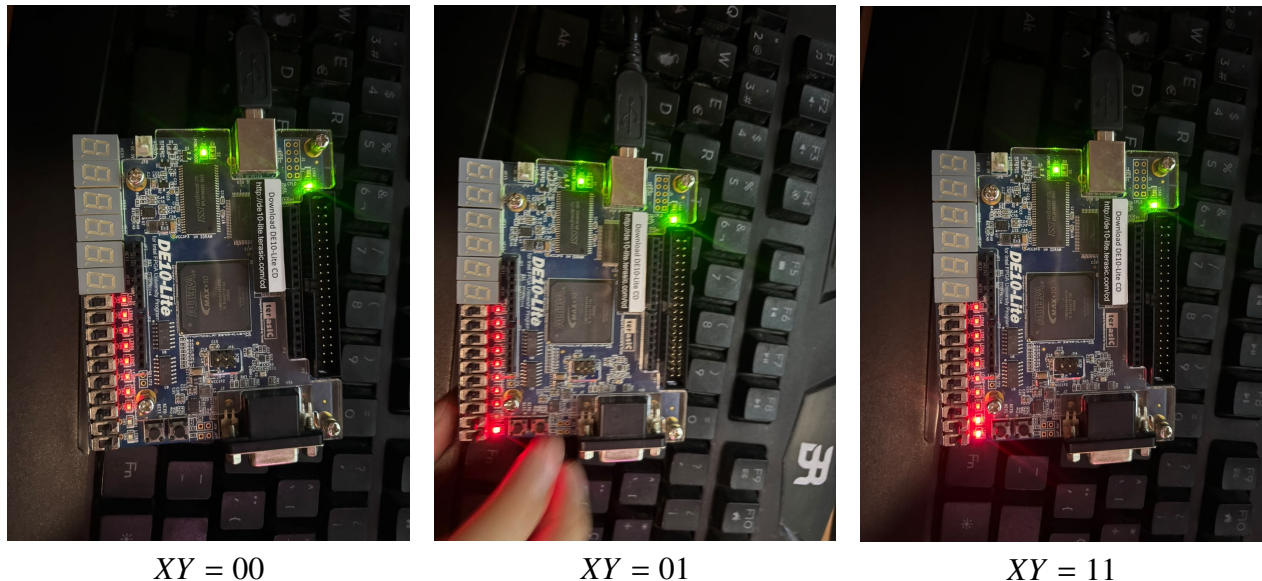


Figura 7: Tarjeta DE10-Lite programada, con los interruptores en las combinaciones $XY = 00, 01$ y 11 .

Las fotografías confirman que la tarjeta quedó programada y encendida (LEDs y *power good* activos) y que responde a los interruptores de entrada. Con la identificación de pines de la tabla 2 ($LEDR0 = S_0$, $LEDR1 = S_1$), el patrón observado en las capturas es congruente con lo esperado: en $XY = 00$ solo LEDR1 está encendido ($S_1S_0 = 10$, EST0); al pasar a $XY = 01$ se observa el cambio a LEDR0 encendido ($S_1S_0 = 01$, EST1); y en $XY = 11$ ambos LEDs permanecen o vuelven a encenderse según la transición correspondiente, en concordancia con la tabla 1.

Análisis de resultados

De acuerdo con las secciones anteriores, el diseño de la Parte I (tabla de transiciones, mapas de Karnaugh y ecuaciones 1-4) se implementó y compiló sin errores en Quartus, y la tarjeta fue programada y responde a los interruptores de entrada.

Con la identificación de pines confirmada ($LEDR0 = S_0$, $LEDR1 = S_1$), el comportamiento observado en la figura 7 es consistente con lo previsto en la tabla 1: en $XY = 00$ la máquina permanece en EST0 ($S_1 S_0 = 10$, es decir LEDR1 encendido y LEDR0 apagado), lo cual coincide con la primera fila de la tabla, donde desde EST0 con $X = 0$ el sistema regresa a EST0. Al cambiar a $XY = 01$ se observa el cambio de patrón a LEDR0 encendido y LEDR1 apagado ($S_1 S_0 = 01$), congruente con la transición EST0→EST1 registrada en la tabla para $X = 1, Y = 1$ y con la salida de EST1. Finalmente, en $XY = 11$ el patrón de LEDs observado corresponde a la permanencia en EST1 o al avance a EST2 según el estado inmediato anterior de la máquina, lo cual también es congruente con las filas correspondientes de la tabla.

No fue posible capturar ni verificar experimentalmente la combinación $XY = 10$, por lo que la comparación completa de las ocho filas de la tabla 1 queda parcial; para las tres combinaciones sí registradas (00, 01, 11) no se encontró ninguna discrepancia entre el comportamiento esperado y el observado en la FPGA.

Conclusiones individuales

Omar Alfonso Zendejas Labias

El objetivo de la práctica se cumplió: a partir de la carta ASM llegamos a la tabla de transiciones, redujimos por Karnaugh las ecuaciones de los flip-flops D y de las salidas, y ese diseño es justo el que terminó compilado en Quartus y programado en la tarjeta. Lo que más se me facilitó fue precisamente esa parte de pasar de la carta ASM a la tabla, ya que al tener bien identificadas las rutas de decisión el llenado fue prácticamente directo. En la tarjeta, para las combinaciones que sí alcanzamos a fotografiar ($XY = 00, 01$ y 11) el comportamiento de los LEDs coincidió con lo que la tabla de la Parte I predecía, así que el diseño quedó validado, aunque nos falta capturar la combinación $XY = 10$ para dejar la comprobación completa. También queda pendiente confirmar con la profesora si capturar la máquina como esquemático en vez de escribirla en VHDL es válido para esta entrega, ya que el manual pide explícitamente esa segunda opción.

Emiliano García Olvera

El desarrollo de esta práctica ayudó a consolidar la relación entre el diseño lógico de una máquina de estados y su implementación física en una FPGA. A nivel teórico, el uso de los mapas de Karnaugh para simplificar las entradas D de los flip-flops tipo D dejó ver cómo agrupaciones aparentemente distintas de la tabla de transiciones pueden combinarse en términos compartidos, como ocurrió con el término $Q_1\overline{Q_0}$ presente tanto en D_1 como en D_0 . A nivel práctico, trabajar con el archivo `div_frec.vhd` proporcionado por la profesora fue útil para entender cómo se integra un módulo VHDL externo con una lógica combinacional/secuencial capturada de forma esquemática dentro del mismo proyecto de Quartus, y cómo ambos comparten la señal de reloj derivada del oscilador de 50 MHz de la tarjeta. La asignación de pines y la posterior programación de la FPGA permitieron cerrar el ciclo completo de diseño: de la especificación abstracta (carta ASM) al circuito físico observable en los LEDs de la DE10-Lite. Como área de mejora para futuras prácticas, sería conveniente planear con anticipación la captura de *todas* las combinaciones de entrada relevantes, para evitar quedarse con evidencia experimental incompleta como ocurrió en este caso con la combinación $XY = 10$.

Referencias

- [1] J. Savage Carmona, G. Vázquez y N. E. Chávez Rodríguez, *Diseño de microprocesadores*. Ciudad de México, México: Facultad de Ingeniería, Universidad Nacional Autónoma de México, 2017. [En línea]. Disponible: <http://profesores.fi-b.unam.mx/luist/archivos/MicroprocesadoresSavage.pdf>